

Modelos de lenguaje

Tabla de contenidos

1	Modelos de lenguaje y LLM	3
1.1	Qué es un modelo de lenguaje	3
1.2	Predecir el siguiente token	4
1.3	Generación autoregresiva	5
1.4	Una tarea sencilla con consecuencias complejas	6
1.5	Predicción no significa recuperación literal	7
1.6	De modelo de lenguaje a gran modelo de lenguaje	7
1.7	Escala y capacidad	8
1.8	Preentrenamiento	9
1.9	Modelos fundacionales	9
1.10	Ajuste para seguir instrucciones	10
1.11	Alineamiento y preferencias humanas	11
1.12	Modelos de propósito general	11
1.13	Aprendizaje en contexto	12
1.14	Capacidades emergentes	13
1.15	Generación probabilística	13
1.15.1	Selección determinista	13
1.15.2	Muestreo	13
1.15.3	Temperatura	14
1.16	La respuesta no estaba escrita de antemano	14
1.17	Conocimiento aprendido y conocimiento verificable	15
1.18	Generar no es verificar	15
1.19	Un LLM no es un sistema inteligente completo	16
1.20	Una primera definición completa	17
1.21	Del modelo al comportamiento	17

1 Modelos de lenguaje y LLM

Un Transformer es una arquitectura capaz de procesar secuencias y construir representaciones contextuales de sus elementos.

Para convertir esa arquitectura en un modelo capaz de trabajar con lenguaje, es necesario definir una tarea de entrenamiento.

Una de las tareas más importantes consiste en predecir qué token debería aparecer a continuación dentro de una secuencia.

Por ejemplo, ante el texto:

El servidor dejó de responder porque...

el modelo podría asignar distintas probabilidades a posibles continuaciones:

estaba	0,24
había	0,18
se	0,13
recibió	0,07
funcionaba	0,04
...	

El modelo no selecciona una palabra mediante una regla escrita previamente.

Calcula una distribución de probabilidad sobre los tokens de su vocabulario y utiliza esa distribución para determinar cuál puede continuar la secuencia.

Esta tarea constituye la base de muchos modelos de lenguaje generativos.

1.1. Qué es un modelo de lenguaje

Un **modelo de lenguaje** es un modelo matemático que representa regularidades presentes en secuencias lingüísticas.

Su objetivo fundamental es estimar la probabilidad de una secuencia o de los elementos que pueden aparecer en ella.

De forma simplificada, un modelo de lenguaje responde a preguntas como:

- ¿qué token es probable después de los anteriores?;
- ¿qué secuencias resultan lingüísticamente plausibles?;

- ¿qué expresiones suelen aparecer en determinados contextos?;
- ¿qué relaciones existen entre conceptos, estructuras y formas de uso?

Un modelo de lenguaje no contiene necesariamente frases completas almacenadas para recuperarlas posteriormente.

Aprende patrones distribuidos entre sus parámetros a partir de los ejemplos utilizados durante el entrenamiento.

Estos patrones pueden incluir regularidades relacionadas con:

- vocabulario;
- gramática;
- sintaxis;
- estilo;
- estructuras textuales;
- relaciones semánticas;
- conocimiento expresado en los datos;
- formas habituales de resolver determinadas tareas.

El modelo aprende estas regularidades porque le ayudan a reducir el error al predecir tokens.

1.2. Predecir el siguiente token

En un modelo autoregresivo, el objetivo consiste en predecir cada token utilizando los tokens anteriores como contexto.

Supongamos la secuencia:

Los sistemas inteligentes combinan modelos y reglas.

Durante el entrenamiento, pueden generarse varios ejemplos:

```
Los
→ sistemas

Los sistemas
→ inteligentes

Los sistemas inteligentes
→ combinan

Los sistemas inteligentes combinan
→ modelos
```

El modelo intenta predecir el token esperado en cada posición.

Cuando su predicción difiere del token real, se calcula un error y se ajustan sus parámetros.

Este proceso se repite sobre enormes cantidades de secuencias.

De forma simplificada:

```
contexto conocido
    ↓
predicción del siguiente token
    ↓
comparación con el token real
    ↓
cálculo del error
    ↓
ajuste de parámetros
```

El objetivo no consiste en memorizar cada frase, sino en desarrollar representaciones y patrones que permitan predecir correctamente secuencias nuevas.

1.3. Generación autoregresiva

Una vez entrenado, el modelo puede generar texto aplicando repetidamente el mismo proceso.

Ante una petición, predice un primer token:

```
contexto inicial → token 1
```

Ese token se añade al contexto:

```
contexto inicial + token 1 → token 2
```

Después se repite la operación:

```
contexto inicial + token 1 + token 2 → token 3
```

La respuesta se construye progresivamente:

contexto → siguiente token → nuevo contexto → siguiente token

Este proceso recibe el nombre de **generación autoregresiva**.

El modelo no escribe primero una respuesta completa para mostrarla después.

La produce token a token, utilizando en cada paso tanto la información inicial como los tokens que ya ha generado.

Esto explica por qué una respuesta puede comenzar correctamente y desviarse posteriormente. Cada nuevo token modifica el contexto utilizado para generar los siguientes.

1.4. Una tarea sencilla con consecuencias complejas

Predecir el siguiente token puede parecer una tarea limitada.

Sin embargo, para realizarla correctamente sobre textos complejos, el modelo debe aprender numerosas regularidades.

Para completar una frase, puede necesitar reconocer:

- la estructura gramatical;
- el significado de los términos;
- el tema del documento;
- referencias anteriores;
- relaciones temporales;
- convenciones de un lenguaje de programación;
- formatos habituales;
- patrones de razonamiento expresados en los datos;
- conocimiento necesario para continuar el contenido.

Consideremos:

Madrid es la capital de...

Para predecir *España*, el modelo necesita haber aprendido una relación presente en los datos de entrenamiento.

En:

Si todos los servidores están apagados y este equipo es un servidor, entonces...

debe reconocer una estructura lógica para generar una continuación coherente.

En:

```
def sumar(a, b):  
    return
```

debe identificar patrones propios del lenguaje de programación.

La tarea de predicción obliga al modelo a construir representaciones útiles de numerosos fenómenos.

Las capacidades observadas no se programan una por una. Aparecen como resultado del aprendizaje de patrones necesarios para mejorar la predicción.

1.5. Predicción no significa recuperación literal

Cuando el modelo genera una respuesta, no suele buscar una frase exacta almacenada en una base de datos interna.

Utiliza sus parámetros para calcular qué continuación resulta probable dentro del contexto actual.

Esto permite:

- combinar conceptos;
- adaptar una explicación;
- modificar el estilo;
- resumir un texto;
- transformar formatos;
- generar ejemplos nuevos;
- producir código;
- responder a instrucciones no vistas exactamente durante el entrenamiento.

Pero esta misma capacidad introduce un riesgo fundamental.

El modelo puede producir una secuencia lingüísticamente coherente aunque su contenido no sea correcto.

Su objetivo básico es generar una continuación probable, no verificar automáticamente que cada afirmación corresponda con la realidad.

1.6. De modelo de lenguaje a gran modelo de lenguaje

Las siglas **LLM** corresponden a *Large Language Model*, o **gran modelo de lenguaje**.

No existe una frontera matemática universal que determine cuándo un modelo pasa a considerarse grande.

El término suele utilizarse para describir modelos caracterizados por una combinación de:

- una gran cantidad de parámetros;
- entrenamiento con grandes volúmenes de datos;
- considerable capacidad de cálculo;
- arquitecturas profundas;

- capacidad para realizar múltiples tareas lingüísticas;
- posibilidad de adaptarse a instrucciones mediante el contexto.

Un LLM sigue siendo un modelo de lenguaje.

La palabra *large* indica su escala, no una naturaleza completamente diferente.

Podemos establecer la siguiente relación:

modelo $\rightarrow$ modelo de lenguaje $\rightarrow$ modelo de lenguaje basado en Transformer $\rightarrow$
gran modelo de lenguaje

1.7. Escala y capacidad

Al aumentar el tamaño del modelo, la cantidad de datos y el cálculo utilizado durante el entrenamiento, suele mejorar su capacidad para representar patrones complejos.

La escala puede permitir:

- mantener relaciones más sofisticadas;
- adaptarse a tareas diversas;
- producir texto más coherente;
- trabajar con distintos estilos y dominios;
- generalizar a instrucciones nuevas;
- realizar tareas sin entrenamiento específico adicional.

Sin embargo, un modelo más grande no es automáticamente mejor para cualquier problema.

También puede implicar:

- mayor coste;
- mayor consumo de recursos;
- más latencia;
- mayores requisitos de infraestructura;
- dificultades de despliegue;
- menor control sobre los datos de entrenamiento;
- comportamientos más complejos de evaluar.

Dentro de un sistema inteligente, la elección del modelo debe depender de la necesidad real.

Utilizar el modelo más grande disponible no constituye por sí mismo una decisión de ingeniería adecuada.

1.8. Preentrenamiento

Los LLM suelen comenzar con una fase denominada **preentrenamiento**.

Durante esta fase, el modelo procesa grandes cantidades de texto y aprende a predecir tokens.

El prefijo *pre-* indica que este entrenamiento se produce antes de adaptar el modelo a tareas o formas de interacción más específicas.

El preentrenamiento proporciona una capacidad general para trabajar con lenguaje.

A partir de él, el modelo puede reconocer y generar:

- estructuras gramaticales;
- diferentes estilos;
- formatos;
- relaciones entre conceptos;
- patrones presentes en código;
- conocimiento expresado en los datos;
- formas habituales de responder o desarrollar argumentos.

El resultado recibe con frecuencia el nombre de **modelo base** o *base model*.

Un modelo base puede continuar texto, pero no necesariamente seguir instrucciones de la forma esperada por un usuario.

1.9. Modelos fundacionales

Un modelo preentrenado de gran escala puede utilizarse como base para múltiples tareas y sistemas posteriores.

Por esta razón, algunos modelos se denominan **modelos fundacionales**.

Un modelo fundacional proporciona capacidades generales que pueden adaptarse mediante:

- instrucciones;
- ejemplos en el contexto;
- entrenamiento adicional;
- ajuste especializado;
- conexión con herramientas;
- recuperación de información;
- integración dentro de aplicaciones.

El término no implica que el modelo constituya por sí mismo la solución final.

Indica que actúa como una base reutilizable sobre la que pueden construirse distintos sistemas.

De nuevo, debemos mantener la distinción central del curso:

el modelo proporciona una capacidad general; el sistema inteligente determina cómo se utiliza.

1.10. Ajuste para seguir instrucciones

Un modelo entrenado únicamente para continuar texto puede producir resultados plausibles, pero no necesariamente comportarse como un asistente.

Para mejorar su capacidad de seguir peticiones, puede realizarse un entrenamiento adicional con ejemplos formados por:

- una instrucción;
- un contexto;
- una respuesta esperada.

Este proceso suele denominarse **ajuste mediante instrucciones** o *instruction tuning*.

Por ejemplo:

```
Instrucción:
Resume el siguiente texto en tres puntos.

Texto:
...

Respuesta esperada:
...
```

El modelo aprende patrones asociados a tareas como:

- resumir;
- clasificar;
- explicar;
- traducir;
- transformar contenido;
- responder preguntas;
- generar código;
- seguir formatos de salida.

El ajuste no sustituye al preentrenamiento.

Parte de las capacidades generales adquiridas previamente y modifica el comportamiento para facilitar la interacción mediante instrucciones.

1.11. Alineamiento y preferencias humanas

Además del ajuste mediante instrucciones, algunos modelos atraviesan procesos destinados a adaptar su comportamiento a preferencias y criterios humanos.

En estos procesos se pueden utilizar:

- evaluaciones de respuestas;
- comparaciones entre alternativas;
- ejemplos corregidos;
- señales de recompensa;
- reglas de comportamiento;
- técnicas de aprendizaje por refuerzo;
- optimización directa a partir de preferencias.

El objetivo es favorecer respuestas que resulten:

- útiles;
- relevantes;
- claras;
- seguras;
- coherentes con las instrucciones;
- adecuadas para el contexto de uso.

Este proceso suele denominarse **alineamiento**.

El término debe utilizarse con precaución. No significa que el modelo comprenda valores humanos ni que su comportamiento esté garantizado en cualquier situación.

Significa que ha sido optimizado para producir determinados comportamientos observables bajo las condiciones evaluadas durante el entrenamiento.

1.12. Modelos de propósito general

Una de las características más relevantes de los LLM es que un mismo modelo puede utilizarse para tareas muy diferentes.

Por ejemplo:

- responder preguntas;
- redactar documentos;
- resumir información;
- extraer datos;
- clasificar contenido;
- transformar formatos;
- generar código;
- explicar conceptos;

- traducir;
- participar en conversaciones.

En el software tradicional, cada una de estas capacidades podría requerir un componente diseñado específicamente.

En un LLM, muchas tareas pueden expresarse mediante instrucciones en lenguaje natural.

Esto convierte al lenguaje en una interfaz de programación de alto nivel.

Sin embargo, la flexibilidad también reduce parte del determinismo habitual del software tradicional.

Una instrucción puede ser ambigua, el resultado puede variar y el modelo puede interpretar la tarea de manera diferente a la esperada.

1.13. Aprendizaje en contexto

Un LLM puede adaptar temporalmente su comportamiento utilizando ejemplos incluidos en el contexto.

Por ejemplo:

```
Entrada: servidor caído
Categoría: infraestructura

Entrada: contraseña olvidada
Categoría: acceso

Entrada: factura incorrecta
Categoría:
```

El modelo puede inferir que debe completar la última categoría siguiendo el patrón de los ejemplos anteriores.

Esta capacidad suele denominarse **aprendizaje en contexto** o *in-context learning*.

El término puede resultar engañoso.

El modelo no modifica necesariamente sus parámetros. Utiliza los ejemplos disponibles en la ventana de contexto para interpretar la tarea y producir una respuesta.

Por tanto:

- el entrenamiento modifica el modelo;
- el aprendizaje en contexto modifica temporalmente la forma de utilizarlo;
- al desaparecer el contexto, desaparecen también esos ejemplos.

1.14. Capacidades emergentes

Algunas capacidades se hacen más visibles al aumentar la escala del modelo, los datos y el entrenamiento.

El modelo puede comenzar a realizar tareas para las que no recibió una programación explícita individual.

Estas capacidades suelen denominarse **emergentes**.

El término no significa que aparezcan de forma mágica.

Describe comportamientos que surgen de la interacción entre:

- la arquitectura;
- los datos;
- el objetivo de entrenamiento;
- el número de parámetros;
- la escala de cálculo;
- el contexto proporcionado.

En algunos casos, una mejora gradual del modelo puede producir un cambio aparentemente brusco en una métrica concreta.

También puede ocurrir que la capacidad existiera parcialmente, pero solo resulte visible al utilizar una forma adecuada de evaluación o de instrucción.

1.15. Generación probabilística

En cada paso, el modelo produce una distribución de probabilidad sobre los posibles tokens siguientes.

La aplicación puede seleccionar el token utilizando diferentes estrategias.

1.15.1. Selección determinista

Se elige el token con mayor probabilidad.

Este enfoque puede producir respuestas más estables, aunque no garantiza que sean siempre idénticas en todos los sistemas.

1.15.2. Muestreo

Se selecciona un token teniendo en cuenta su probabilidad.

Esto permite mayor variedad, pero también introduce más variabilidad.

1.15.3. Temperatura

La **temperatura** modifica la distribución utilizada durante la selección.

Con una temperatura baja, el sistema favorece los tokens más probables.

Con una temperatura más alta, aumenta la posibilidad de seleccionar alternativas menos probables.

De forma conceptual:

- temperatura baja: respuestas más conservadoras;
- temperatura alta: respuestas más variadas;
- temperatura muy alta: mayor riesgo de incoherencia.

La temperatura no cambia los parámetros del modelo ni su conocimiento.

Modifica la forma en que se seleccionan los tokens durante la generación.

1.16. La respuesta no estaba escrita de antemano

La salida de un LLM se construye durante la inferencia.

No existe necesariamente una respuesta completa almacenada esperando a ser recuperada.

Cada token depende de:

- los parámetros del modelo;
- la petición;
- el contexto;
- los tokens ya generados;
- la estrategia de selección;
- la configuración del sistema.

Por esta razón, una misma petición puede producir respuestas diferentes.

También explica por qué pequeñas modificaciones en la entrada pueden alterar significativamente el resultado.

El modelo trabaja con relaciones probabilísticas dentro de una secuencia, no con una función determinista diseñada manualmente para cada tarea.

1.17. Conocimiento aprendido y conocimiento verificable

Durante el entrenamiento, el modelo incorpora patrones relacionados con la información presente en los datos.

En ese sentido, puede responder utilizando conocimiento adquirido durante el preentrenamiento.

Sin embargo, este conocimiento presenta limitaciones:

- puede estar incompleto;
- puede estar desactualizado;
- puede reflejar errores de los datos;
- puede combinar fuentes incompatibles;
- puede no conservar detalles exactos;
- puede producir afirmaciones plausibles pero falsas.

Un LLM no debe considerarse una base de datos exacta.

Sus parámetros representan regularidades distribuidas, no una colección perfectamente indexada de hechos verificables.

Cuando la precisión sea importante, el sistema inteligente debe complementar el modelo con:

- fuentes externas;
- bases de datos;
- documentos;
- herramientas;
- mecanismos de recuperación;
- validaciones;
- revisión humana.

1.18. Generar no es verificar

Un LLM puede producir una respuesta bien redactada sin haber comprobado su contenido.

Esta diferencia es esencial:

- **generar** consiste en producir una secuencia probable;
- **verificar** consiste en contrastarla con criterios, datos o fuentes fiables.

El modelo puede generar:

- una fecha incorrecta;
- una referencia inexistente;
- un cálculo erróneo;
- una interpretación jurídica inadecuada;

- una función de programación defectuosa;
- una explicación convincente pero falsa.

La fluidez de la respuesta no demuestra su corrección.

Por ello, el diseño del sistema debe decidir qué resultados pueden utilizarse directamente y cuáles necesitan validación.

1.19. Un LLM no es un sistema inteligente completo

Un LLM puede aportar numerosas capacidades:

- interpretación de lenguaje;
- generación de contenido;
- clasificación;
- extracción;
- transformación;
- razonamiento sobre el contexto;
- selección de acciones propuestas.

Pero no constituye necesariamente una solución completa.

Un sistema inteligente puede necesitar además:

- obtener información actualizada;
- almacenar estado;
- aplicar reglas;
- controlar permisos;
- utilizar herramientas;
- ejecutar acciones;
- comprobar resultados;
- registrar operaciones;
- gestionar errores;
- mantener trazabilidad;
- solicitar supervisión humana.

Podemos representar esta relación así:

```
sistema inteligente
  modelo de lenguaje
  instrucciones
  contexto
  datos
  memoria
  herramientas
  reglas
  validaciones
```


El LLM es una pieza especialmente versátil, pero sigue siendo una pieza.

1.20. Una primera definición completa

Podemos definir un LLM como:

Un modelo de lenguaje de gran escala, normalmente basado en una arquitectura Transformer, entrenado con grandes cantidades de datos para predecir tokens y capaz de adaptarse mediante instrucciones y contexto a múltiples tareas relacionadas con el lenguaje.

Esta definición contiene los elementos estudiados hasta ahora:

- es un **modelo**;
- trabaja con **lenguaje**;
- representa el texto mediante **tokens**;
- utiliza **embeddings**;
- relaciona la secuencia mediante **atención**;
- suele utilizar una arquitectura **Transformer**;
- adquiere sus parámetros mediante **entrenamiento**;
- genera resultados mediante **inferencia**;
- produce texto de forma **probabilística**;
- utiliza la información disponible en el **contexto**.

1.21. Del modelo al comportamiento

Ya conocemos las piezas fundamentales que permiten construir un gran modelo de lenguaje.

El siguiente paso consiste en comprender qué consecuencias tiene su forma de funcionamiento.

Un LLM genera secuencias probables, trabaja con información limitada por su contexto y utiliza representaciones aprendidas a partir de datos.

Estas características explican tanto sus capacidades como sus limitaciones.

En la siguiente sección analizaremos conceptos como:

- probabilidad;
- variabilidad;
- generalización;
- errores;

- alucinaciones;
- sesgos;
- razonamiento;
- confianza;
- verificación.

Comprender estos límites será imprescindible antes de utilizar el modelo como componente de un sistema inteligente.